FRONTEND DEVELOPMENT / ASSIGNMENT 6

Foodly

jQuery: Search, UX Engagement and Web App Functionality

Team name: KN (Foodly)
Members: Kakhar Nassyrkhan and Nurasyl Raiskaliev
Group: SE-2506

Website and repository - clickable links

LIVE WEBSITE
https://vuus1d.github.io/Food-Delivery-Platform-KN/

GITHUB REPOSITORY
https://github.com/vuus1d/Food-Delivery-Platform-KN

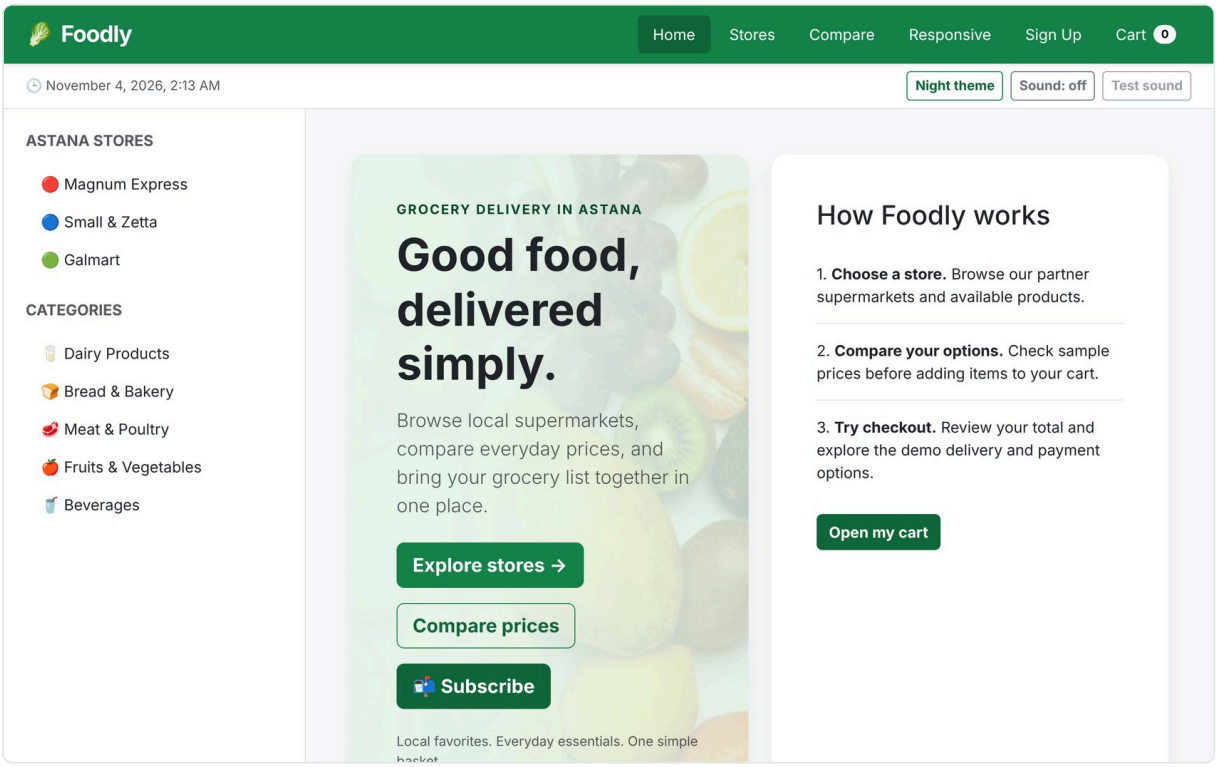


Figure 1. The Foodly homepage after Assignment 6, captured from the working implementation.

Objective

Continue the Week 2 food-delivery project and use jQuery to add real-time search, autocomplete and search highlighting, engaging UX elements (scroll progress, animated counters, loading spinners) and app features (toast notifications, copy-to-clipboard and lazy image loading), then redeploy the site.

This is an educational frontend. Prices, promo codes, statistics and orders are demonstration data; no real account, email, payment or delivery is created. Screenshots use synthetic test data.

Requirements and chosen implementation

Every task of the Assignment 6 PDF is implemented in the existing Foodly pages. All new jQuery code is in one external file, `js/script.js` (594 lines), divided into sections titled `/* ===== Task N ===== */`. New styles are in the `Assignment 6` block of `css/style.css`.

Requirement	Selected implementation	Page
0. Setup	jQuery 4.0.0 from the CDN in <head>; `js/script.js` with `$(document).ready()`.	3
1. Real-time search & live filter	Product catalog on Stores and price table on Compare filter on `keyup input` with `.filter()`.	4
2. Autocomplete suggestions	Dropdown of matching products under the catalog search; mouse and keyboard.	5
3. Search highlighting	FAQ search wraps every match in `<mark>`; matching answers open.	6
4. Scroll progress bar	Gradient bar with glow on all pages; day and night variants.	7
5. Animated number counter	“Foodly in numbers” stats count up once when visible.	8
6. Loading spinner on submit	Feedback, subscription, registration and checkout buttons.	9-10
7. Notification system	Stacked jQuery toasts for cart, forms, orders and copying.	11
8. Copy to clipboard	Promo codes and demo order number; checkmark, tooltip, `.on('copy')`.	12-13
9. Image lazy loading	13 images wait in `data-src`; loaded by `.offset()` on scroll and by carousel events.	14-15
Verification	Independent browser re-check and the project test suite.	16
Part 4. Deploy	Commit, push to main, GitHub Pages rebuild and live check.	17
Report and defense	Code/result screenshots, steps, verification, reflection.	All; 18

The previous features (day/night theme, cart, three-step checkout, form validation, sound, keyboard navigation) were kept and re-tested. The old vanilla catalog filter and toast in `app.js` were replaced by the jQuery versions, so no element has two competing handlers.

0. Setup

1. Include jQuery 4.0.0 from `code.jquery.com` inside <head> on all six pages, with an SRI `integrity` hash.
2. Load our external `js/script.js` right after it (not deferred), so `$` exists for every later script.
3. Wrap start-up code in `$(document).ready()`, print “jQuery is ready!” and call each task's `init…()` function.

index.html · lines 11-13

```
11      <!-- Assignment 6, Task 0: jQuery 4.0.0 from the official CDN, then our jQuery code (js/script.js) -->
12      <script src="https://code.jquery.com/jquery-4.0.0.min.js" integrity="sha256-OaVG6prZf4v69dPg6PhVattBXkcOWQB62pdZ3ORyrao="
    crossorigin="anonymous"></script>
13      <script src="js/script.js"></script>
```

Figure 2. Actual source: index.html <head> (the same lines are on all six pages).

js/script.js · lines 584-594

```
584 $(document).ready(function () {
585     console.log("jQuery is ready!");
586     initScrollProgress();
587     initCatalogSearch();
588     initAutocomplete();
589     initCompareSearch();
590     initFaqSearch();
591     initCounters();
592     initCopyButtons();
593     initLazyImages();
594 });
```

Figure 3. Actual source: js/script.js, $(document).ready() starts every Assignment 6 feature.

Browser console · captured with Playwright (Chromium) on every page

```
1 index.html  →  jQuery 4.0.0 · [log] jQuery is ready!
2 stores.html  →  jQuery 4.0.0 · [log] jQuery is ready!
3 compare.html  →  jQuery 4.0.0 · [log] jQuery is ready!
4 cart.html  →  jQuery 4.0.0 · [log] jQuery is ready!
5 login.html  →  jQuery 4.0.0 · [log] jQuery is ready!
6 responsive.html  →  jQuery 4.0.0 · [log] jQuery is ready!
```

Figure 4. Console output recorded on each page: jQuery 4.0.0 is loaded and “jQuery is ready!” is printed, with 0 errors.

jQuery used: `$(document).ready()`, `$()`

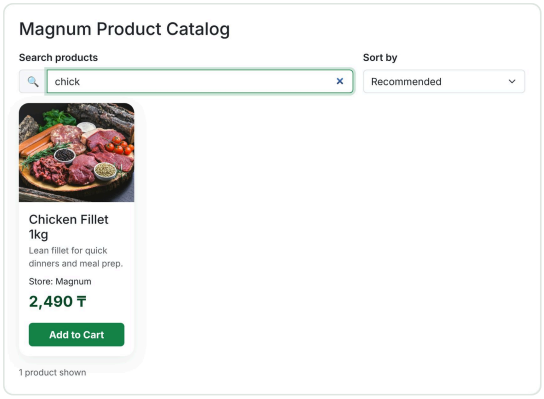
1. Real-time search and live filter

1. Listen to `keyup` and `input` on `#catalog-search`; `input` also catches paste and the × button.
2. Sort the cards with a `switch` on the selected option, then `.append()` moves them into the new order.
3. `.filter()` keeps the cards whose name or description contains the query; the rest get `hidden`, matches fade in.
4. Update “N products shown” and the empty-state message. Compare uses the same filter combined with category buttons.

js/script.js · lines 97-138

```
97  function filterCatalog() {
98      const $container = $("#products");
99      const $search = $("#catalog-search");
100     if (!$container.length || !$search.length) return;
101
102     const query = $search.val().trim().toLowerCase();
103     const sortBy = $("#catalog-sort").val();
104     const priceOf = card => Number($(card).find(".add-product").attr("data-price"));
105     const nameOf = card => $(card).find("h3").text();
106
107     // 1. Sort with a switch, then .append() moves the same card elements into the new order.
108     const sortedCards = $container.children().get().sort((first, second) => {
109         switch (sortBy) {
110             case "price-asc": return priceOf(first) - priceOf(second);
111             case "price-desc": return priceOf(second) - priceOf(first);
112             case "name": return nameOf(first).localeCompare(nameOf(second));
113             default: return Number($(first).attr("data-order")) - Number($(second).attr("data-order"));
114         }
115     });
116     $container.append(sortedCards);
117
118     // 2. .filter() keeps the cards whose name or description contains the query.
119     const $cards = $container.children();
120     const $matches = $cards.filter(function () {
121         return $(this).find("h3, .product-description").text().toLowerCase().includes(query);
122     });
123     $cards.not($matches).prop("hidden", true);
124     revealWithFade($matches);
125
126     // 3. Update the counter and the "No products match" message.
127     const count = $matches.length;
128     $("#catalog-empty").prop("hidden", count !== 0);
129     $("#catalog-status").text(`${count} product${count === 1 ? "" : "s"} shown`);
130     checkLazyImages(); // cards that became visible may need their photos
131 }
132
133 function initCatalogSearch() {
134     // "input" catches typing, paste and the search field's × button; "keyup" is the classic key event.
135     // Both may fire for one key press, which is harmless because filtering is idempotent.
136     $("#catalog-search").on("keyup input", filterCatalog);
137     $("#catalog-sort").on("change", filterCatalog);
138 }
```

Figure 5. Actual source: js/script.js, filterCatalog() and its event listeners.



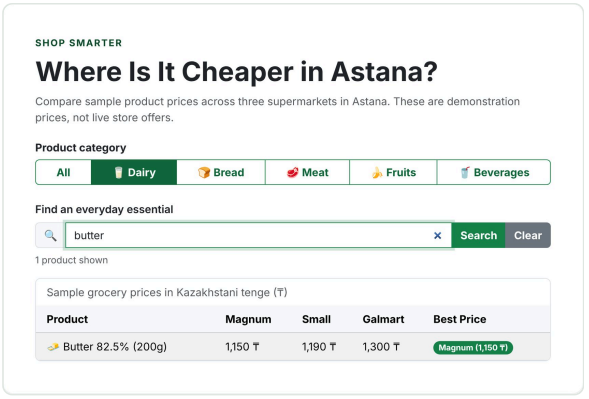


Figure 6. Left: Magnum catalog filtered to 1 of 8 cards for “chick”. Right: Compare page, Dairy + “butter” leaves 1 row.

jQuery used: `.on('keyup input')`, `.val()`, `.filter()`, `.not()`, `.prop()`, `.append()`, `.fadeIn()`, `.text()`

2. Autocomplete search suggestions

1. Collect product names and prices from the rendered catalog with `.map().get()`.
2. On every `input`, keep up to six names containing the query (names that start with it first) and rebuild the list.
3. Build each option from three `.text()` pieces, so the match is in `<mark>` and typed HTML is never parsed.
4. Arrow keys move the active option (`aria-activedescendant`), Enter or a click fills the field and filters, Escape or a click outside closes it.

js/script.js · lines 216-253

```
216     function renderSuggestions() {
217         const query = $input.val().trim();
218         const lowerQuery = query.toLowerCase();
219         if (!query) {
220             closeList();
221             return;
222         }
223         // Matching names, the ones that start with the query first.
224         const matches = catalogItems()
225             .filter(item => item.name.toLowerCase().includes(lowerQuery))
226             .sort((a, b) => a.name.toLowerCase().indexOf(lowerQuery) - b.name.toLowerCase().indexOf(lowerQuery))
227             .slice(0, MAX_SUGGESTIONS);
228         if (!matches.length) {
229             closeList();
230             $status.text("No suggestions.");
231             return;
232         }
233
234         $list.empty();
235         matches.forEach((item, index) => {
236             const start = item.name.toLowerCase().indexOf(lowerQuery);
237             const end = start + query.length;
238             // Three .text() pieces: before, <mark>match</mark>, after. Nothing is parsed as HTML.
239             const $name = $('<span class="suggestion-name"></span>').append(
240                 $("<span></span>").text(item.name.slice(0, start)),
241                 $("<mark></mark>").text(item.name.slice(start, end)),
242                 $("<span></span>").text(item.name.slice(end))
243             );
244             $("<li></li>", { id: `catalog-suggestion-${index}`, role: "option", "aria-selected": "false", class: "autocomplete-option" })
245                 .data("value", item.name)
246                 .append($name, $('<span class="suggestion-price"></span>').text(formatPrice(item.price)))
247                 .appendTo($list);
248         });
249         $list.prop("hidden", false);
250         $input.attr("aria-expanded", "true");
251         setActive(-1);
252         $status.text(`${matches.length} suggestion${matches.length === 1 ? "" : "s"}. Use the up and down arrows to choose.`);
253     }
```

Figure 7. Actual source: js/script.js, renderSuggestions() builds the dropdown safely.

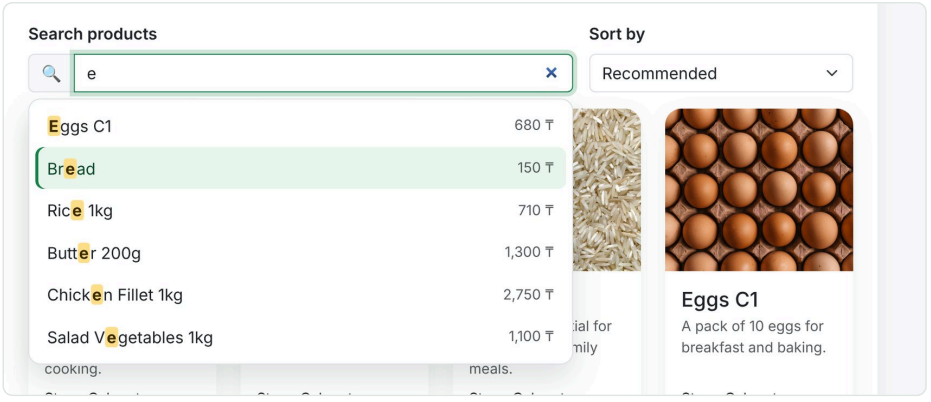


Figure 8. Galmart catalog: typing “e” shows six suggestions with the match highlighted; ArrowDown selected “Bread”.

jQuery used: `.on('input keydown')`, `.map().get()`, `$('<li>', {...})`, `.data()`, `.empty()`, `.eq()`, `.attr()`, `.closest()`

3. Search highlighting

1. Save the original HTML of every FAQ question and answer with `.data('originalHtml', $(this).html())`.
2. On each input, restore that HTML, escape the query (`escapeRegExp`) and build a case-insensitive global regex.
3. Walk only text nodes with `.contents()`, split them by the regex and `.wrap('<mark>')` the matches, so links stay intact.
4. Open answers that contain a match through the existing accordion and show “Found N matches”; clearing restores everything.

js/script.js · lines 312-326

```
312 function highlightMatches(element, pattern) {
313     $(element).contents().each(function () {
314         if (this.nodeType === Node.ELEMENT_NODE) {
315             highlightMatches(this, pattern);
316             return;
317         }
318         if (this.nodeType !== Node.TEXT_NODE) return;
319         // split() with a capturing group keeps the matches at the odd positions.
320         const parts = this.nodeValue.split(pattern);
321         if (parts.length < 2) return;
322         const textNodes = parts.map(part => document.createTextNode(part));
323         $(this).replaceWith(textNodes);
324         $(textNodes.filter((node, index) => index % 2 === 1)).wrap('<mark class="search-hit"></mark>');
325     });
326 }
```

Figure 9. Actual source: js/script.js, highlightMatches() wraps matches inside text nodes only.

js/script.js · lines 342-360

```
342     $input.on("input", function () {
343         const query = $(this).val().trim();
344         // 1. Restore the original markup: removes the old <mark> tags (our own trusted HTML).
345         $targets.each(function () { $(this).html($(this).data("originalHtml")); });
346 
347         if (!query) {
348             // 2a. Cleared: close only the answers that the search opened.
349             $items.each(function () {
350                 const $item = $(this);
351                 if ($item.data("openedBySearch")) setOpen($item, false);
352                 $item.removeData("openedBySearch");
353             });
354             $status.text("");
355             return;
356         }
357 
358         // 2b. Escaped, case-insensitive ("i") pattern that finds every match ("g").
359         const pattern = new RegExp(`(${escapeRegExp(query)})`, "gi");
360         $targets.each(function () { highlightMatches(this, pattern); });
```

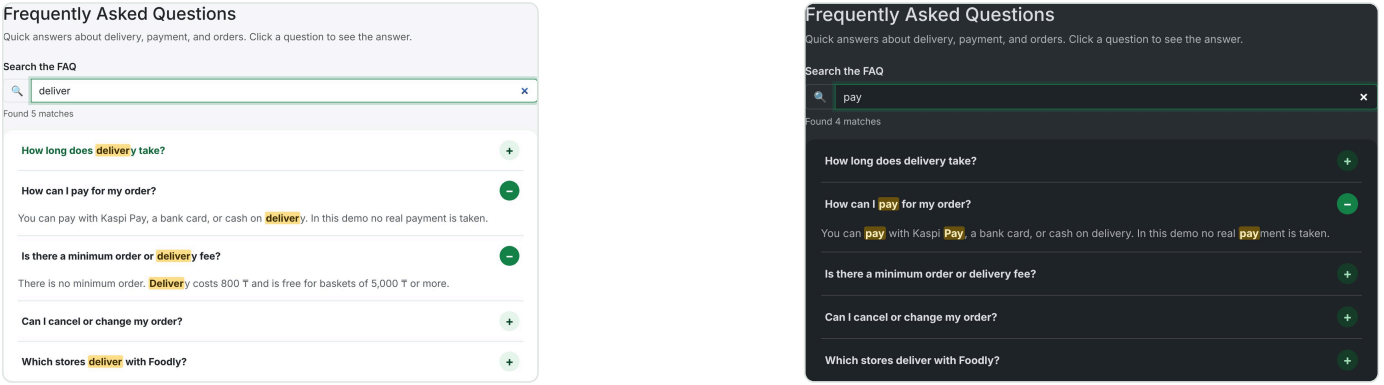


Figure 10. “deliver” gives 5 matches and opens 2 answers (left); “pay” in night theme (right).

jQuery used: `.data()`, `.html()`, `.contents()`, `.each()`, `.replaceWith()`, `.wrap()`, `.trigger('click')`

4. Colorful scroll progress bar

1. Create the bar with jQuery and `.prependTo('body')`; it is decorative, so it has `aria-hidden`.
2. On `scroll` and `resize`, percent = `$(window).scrollTop()` / (`$(document).height()` − `$(window).height()`) × 100.
3. Set the width with `.css()`; hide the track when the page cannot scroll.
4. Style: fixed 6 px bar, Foodly gradient from dark green to orange revealed as you scroll, soft glow, rounded end; brighter night variant.

js/script.js · lines 382-397

```
382 function initScrollProgress() {
383     const $track = $('<div class="scroll-progress" aria-hidden="true"><div class="scroll-progress-bar"></div></div>').prependTo("body");
384     const $bar = $track.children();
385     const $window = $(window);
386
387     function updateProgress() {
388         // How far the page can scroll, and how much of that has been scrolled.
389         const scrollable = $(document).height() - $window.height();
390         const percent = scrollable > 0 ? Math.min(100, ($window.scrollTop() / scrollable) * 100) : 0;
391         $bar.css("width", `${percent}%`);
392         $track.toggleClass("is-idle", scrollable <= 0); // nothing to scroll: hide the track
393     }
394
395     $window.on("scroll resize", updateProgress);
396     updateProgress();
397 }
```

Figure 11. Actual source: js/script.js, initScrollProgress().

css/style.css · lines 253-268

```
253 .scroll-progress { position: fixed; top: 0; left: 0; right: 0; z-index: 1030; height: 6px; background: rgba(22, 131, 75, 0.12); pointer-
    events: none; }
254 .scroll-progress.is-idle { display: none; }
255 .scroll-progress-bar {
256     width: 0;
257     height: 100%;
258     border-radius: 0 999px 999px 0;
259     background: linear-gradient(90deg, #0b4a2a 0%, #16834b 25%, #2fbf71 50%, #f5c542 78%, #ff7a3d 100%);
260     background-size: 100vw 100%;
261     box-shadow: 0 0 10px rgba(47, 191, 113, 0.7), 0 0 3px rgba(245, 197, 66, 0.9);
262     transition: width 80ms linear;
263 }
264 [data-bs-theme="dark"] .scroll-progress { background: rgba(255, 255, 255, 0.08); }
265 [data-bs-theme="dark"] .scroll-progress-bar {
266     background-image: linear-gradient(90deg, #2fbf71 0%, #7be3a6 35%, #ffd166 70%, #ff8c5a 100%);
267     box-shadow: 0 0 12px rgba(123, 227, 166, 0.85), 0 0 4px rgba(255, 209, 102, 0.9);
268 }
```

Figure 12. Actual source: css/style.css, bar styles for day and night theme.

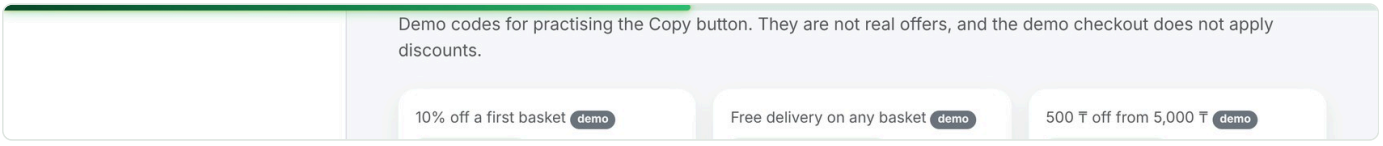


Figure 13. Home page scrolled half-way: the gradient bar at the top edge is exactly 50% wide (measured).

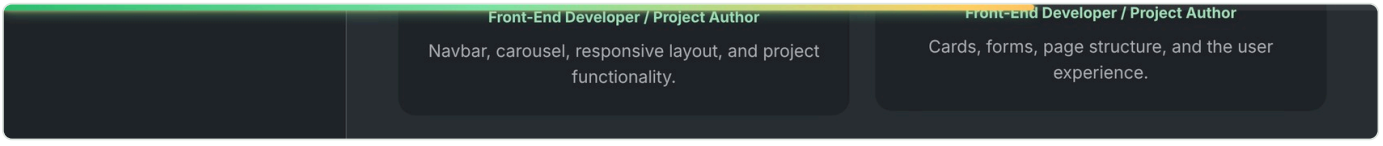


Figure 14. Night theme at 75%: the warmer end of the gradient appears, with a brighter glow.

jQuery used: `$(window).scrollTop()`, `$(document).height()`, `$(window).height()`, `.css()`, `.toggleClass()`, `.prependTo()`

5. Animated number counter

1. Add a “Foodly in numbers” section; every `.counter` stores its target in `data-count-to` and a suffix.
2. Set all counters to 0, then check on scroll whether the section's `.offset().top` is inside the viewport.
3. When visible, remove the namespaced handlers (`.off('scroll.counters')`) so counting runs only once.
4. `$({value: 0}).animate({value: target})` with a custom ease-out; `step` prints each frame, `complete` sets the exact value.

js/script.js · lines 401-432

```
401 function initCounters() {
402     const $section = $("#stats");
403     if (!$section.length) return;
404     const $window = $(window);
405     const $counters = $section.find(".counter");
406     const formatCount = ($counter, value) => `${$counter.attr("data-prefix") || ""}${Math.round(value).toLocaleString("en-
    US")}${$counter.attr("data-suffix") || ""}`;
407
408     // Start from zero; the real values from the HTML stay in the visually hidden text for screen readers.
409     $counters.each(function () { $(this).text(formatCount($(this), 0)); });
410
411     function startWhenVisible() {
412         const sectionTop = $section.offset().top;
413         const viewBottom = $window.scrollTop() + $window.height();
414         if (viewBottom < sectionTop + 80) return; // not visible yet
415         $window.off("scroll.counters resize.counters"); // count only once
416
417         $counters.each(function () {
418             const $counter = $(this);
419             const target = Number($counter.attr("data-count-to"));
420             // .animate() on a plain object: jQuery tweens "value" and step() prints each frame.
421             $({ value: 0 }).animate({ value: target }, {
422                 duration: 1800,
423                 easing: "foodlyEaseOut",
424                 step: now => $counter.text(formatCount($counter, now)),
425                 complete: () => $counter.text(formatCount($counter, target))
426             });
427         });
428     }
429
430     $window.on("scroll.counters resize.counters", startWhenVisible);
431     startWhenVisible();
432 }
```

Figure 15. Actual source: js/script.js, initCounters().

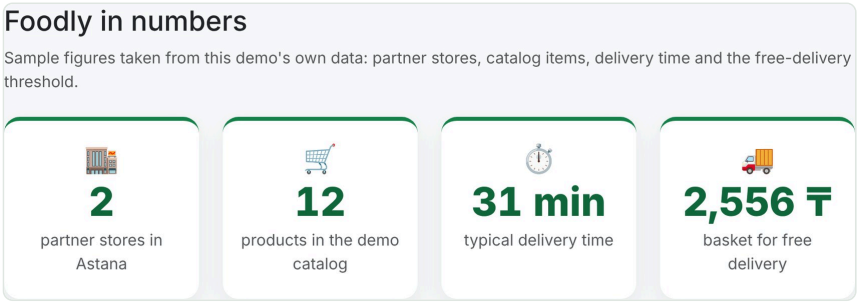


Figure 16. Captured 0.38 s after the section scrolled into view: the counters are still counting.

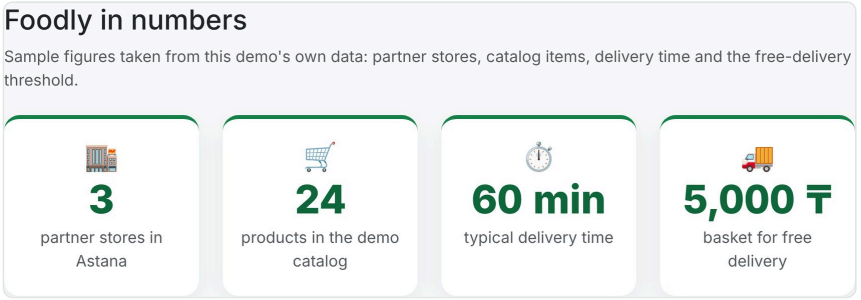


Figure 17. Final values: 3 stores, 24 catalog products, 60 min, 5,000 ₸ free-delivery threshold.

The figures come from the demo's own data and are labelled as sample figures on the page. Screen readers get the final value from visually hidden text.

jQuery used: `.animate() with step/complete`, `$.easing`, `.offset()`, `.on()/.off() with namespaces`, `.text()`

6. Loading spinner on submit

1. The spinner runs only after validation passes: `FoodlyValidation.attach()` calls the success callback for valid forms only.
2. `simulateServerRequest()` saves the button HTML, keeps its width and sets `disabled`, `aria-busy` and spinner + “Please wait…”.
3. After 1.6 s (the simulated server call) the original button is restored and the success message and toast appear.
4. Checkout also disables Back/Cancel and blocks closing the dialog until the simulated response arrives.

js/script.js · lines 436-452

```
436 const SERVER_DELAY = 1600; // simulated server response time, ms
437
438 // Called by validation.js / checkout.js only AFTER validation has passed.
439 function simulateServerRequest(button, onDone) {
440     const $button = $(button);
441     if ($button.prop("disabled")) return;
442     const originalHtml = $button.html(); // our own button markup, restored later
443     $button.css("min-width", $button.outerWidth()) // keep the width while the label changes
444         .prop("disabled", true)
445         .attr("aria-busy", "true")
446         .html('<span class="spinner-border spinner-border-sm me-2" aria-hidden="true"></span>Please wait…');
447
448     setTimeout(() => {
449         $button.html(originalHtml).prop("disabled", false).removeAttr("aria-busy").css("min-width", "");
450         onDone();
451     }, SERVER_DELAY);
452 }
```

Figure 18. Actual source: js/script.js, simulateServerRequest().

js/validation.js · lines 118-128

```
118         FoodlyValidation.attach(form, values => {
119             const status = document.getElementById(statusId);
120             const text = message(values);
121             // Assignment 6, Task 6: validation passed, so the button shows a spinner while a
122             // server call is simulated (js/script.js). The success message and toast come after it.
123             simulateServerRequest(form.querySelector('[type="submit"]'), () => {
124                 status.textContent = text;
125                 status.hidden = false;
126                 showToast(toastText, "success");
127             });
128         });
```

Figure 19. Actual source: js/validation.js, the spinner is started from the success callback.

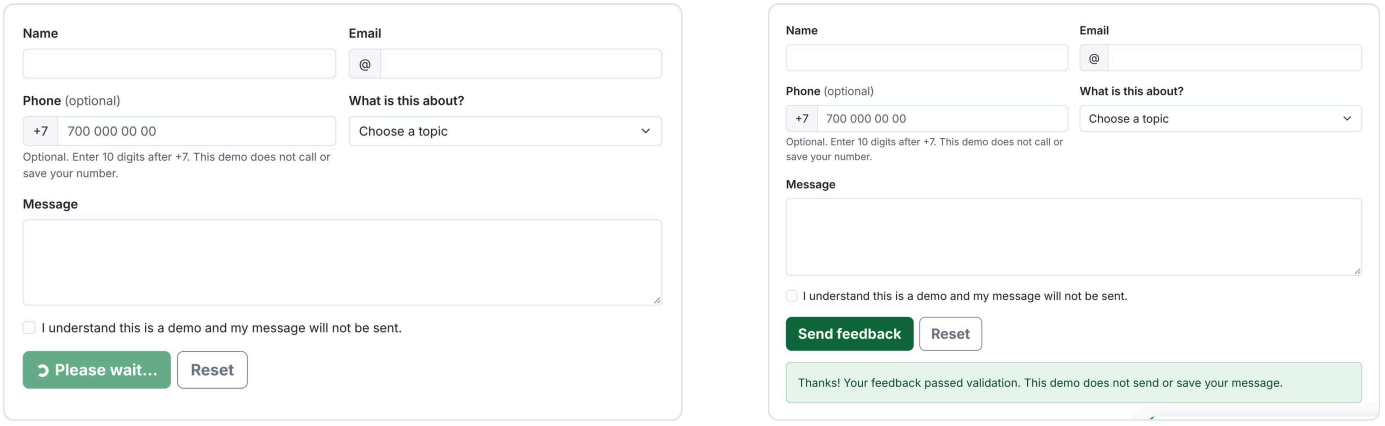


Figure 20. Feedback form: valid submit shows the spinner and a disabled button (left); 1.6 s later the success message (right).

6. Loading spinner on submit (continued)

js/checkout.js · lines 104-120

```
104         // Assignment 6, Task 6: spinner on "Place demo order" while a server call is simulated.
105         // The dialog cannot be closed until the simulated response arrives.
106         setPending(true);
107         simulateServerRequest(submit, () => {
108             setPending(false);
109             completeOrder();
110         });
111     });
112
113     function setPending(pending) {
114         checkout.pending = pending;
115         back.disabled = pending;
116         dialog.querySelectorAll('[data-bs-dismiss="modal"]').forEach(button => { button.disabled = pending; });
117     }
118     dialog.addEventListener("hide.bs.modal", event => {
119         if (checkout.pending) event.preventDefault();
120     });
```

Figure 21. Actual source: js/checkout.js, spinner on “Place demo order” and a locked dialog.

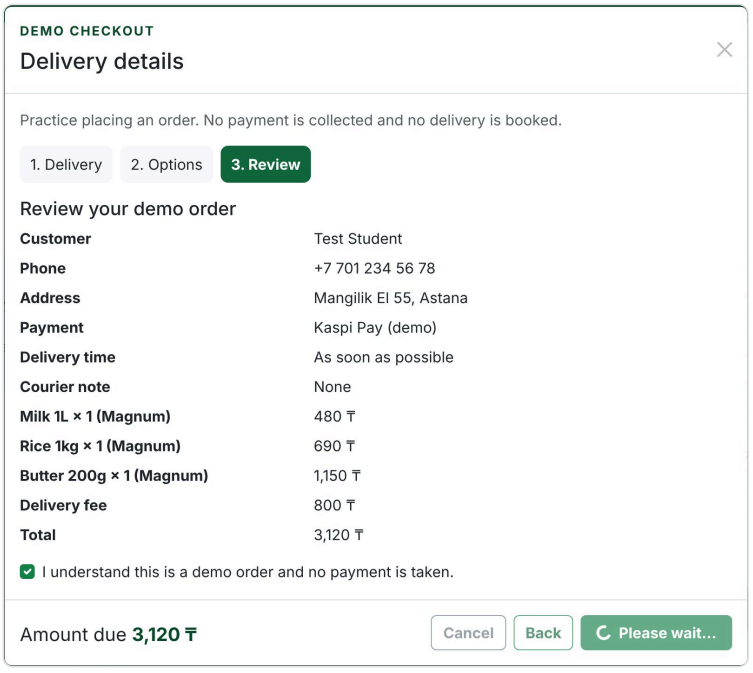


Figure 22. Checkout review step: “Place demo order” shows the spinner; Back and Cancel are disabled.

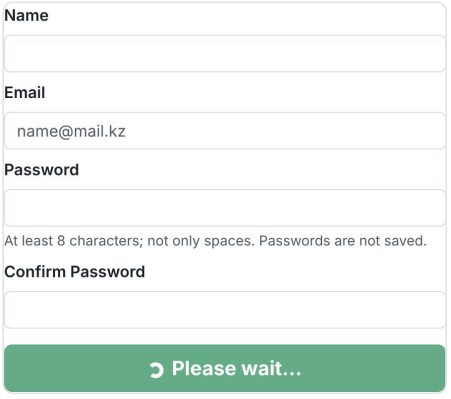


Figure 23. Sign Up page: the “Create account” button during the simulated request.

An invalid submit shows field errors and no spinner (checked automatically). Repeated clicks are impossible because the button is disabled.

jQuery used: `.html()`, `.prop('disabled')`, `.attr()/.removeAttr('aria-busy')`, `.css()`, `.outerWidth()`

7. Notification system

1. One function, `showToast(message, type)`, creates the stack once with `.appendTo('body')`.
2. The toast is built with jQuery: type icon, message via `.text()`, close button, `role="status"`.
3. It slides in and fades in with `.animate({opacity: 1, left: 0})`; after 3.5 s `.fadeOut()` removes it.
4. Hover or focus pauses the timer; at most four toasts are active. Used for cart, forms, orders and copying.

js/script.js · lines 51-92

```
51  function dismissToast($toast) {
52      if ($toast.hasClass("is-closing")) return;
53      $toast.addClass("is-closing"); // still fading out, no longer counted in the stack
54      clearTimeout($toast.data("timer"));
55      $toast.fadeOut(300, () => $toast.remove());
56  }
 …  // lines 57–62 omitted
63  function showToast(message, type = "success") {
64      const kind = Object.hasOwn(TOAST_ICONS, type) ? type : "info";
65      let $stack = $("#toast-stack");
66      if (!$stack.length) {
67          $stack = $('<div id="toast-stack" class="toast-stack toast-container position-fixed bottom-0 end-0 p-3">
    </div>').appendTo("body");
68      }
69      // The oldest toast leaves first, so the stack never covers the page.
70      const $current = $stack.children(".foodly-toast:not(.is-closing)");
71      if ($current.length >= MAX_TOASTS) dismissToast($current.first());
72
73      // .text() inserts the message as plain text, never as HTML.
74      const $toast = $('<div class="toast show foodly-toast" role="status" aria-atomic="true"></div>').addClass(`toast-${kind}`);
75      const $close = $('<button type="button" class="btn-close me-2 m-auto" aria-label="Close notification"></button>');
76      $toast.append(
77          $('<div class="d-flex align-items-center"></div>').append(
78              $('<span class="toast-icon" aria-hidden="true"></span>').text(TOAST_ICONS[kind]),
79              $('<div class="toast-body"></div>').text(message),
80              $close
81          )
82      );
83
84      // Slide in from the right while fading in, then fade out automatically.
85      $toast.css({ opacity: 0, left: "2rem" }).appendTo($stack).animate({ opacity: 1, left: 0 }, 250);
86      $close.on("click", () => dismissToast($toast));
87      // Pause while the pointer or keyboard focus is on the toast, so it can be read.
88      $toast.on("mouseenter focusin", () => clearTimeout($toast.data("timer")))
89          .on("mouseleave focusout", () => startToastTimer($toast));
90      startToastTimer($toast);
91      return $toast;
92  }
```

Figure 24. Actual source: js/script.js, dismissToast() and showToast().

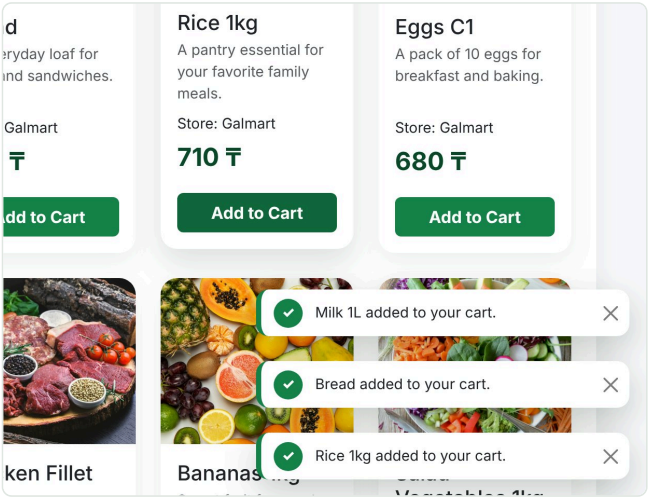


Figure 25. Stores page: three products added in a row produce three stacked toasts.

jQuery used: `.appendTo()`, `.append()`, `.text()`, `.animate()`, `.fadeOut()`, `.remove()`, `.on('mouseenter focusin')`, `.data()`

8. Copy to clipboard button

1. Add a “Weekly promo codes” block (demo codes) and a demo order number after checkout, each with a Copy button.
2. A delegated `$(document).on('click', '.copy-btn')` reads the text from `data-copy-target`.
3. `navigator.clipboard.writeText()` copies it; a hidden textarea + `execCommand('copy')` is the fallback.
4. The icon changes to a checkmark, a “Copied to clipboard!” tooltip is placed with `.offset()`, a toast confirms; all resets after 2 s.
5. `$(document).on('copy', '.copy-snippet-code')` also reacts when a user selects a code and presses Ctrl+C.

js/script.js · lines 474-521

```
474 function copyText(text) {
475     if (navigator.clipboard && window.isSecureContext) {
476         return navigator.clipboard.writeText(text).catch(() => copyWithTextarea(text));
477     }
478     return copyWithTextarea(text);
479 }
480
  … // lines 481–504 omitted
505 function initCopyButtons() {
506     $(".copy-btn .copy-icon").html(ICON_COPY);
507
508     // Delegated, so buttons added later (e.g. the order number on cart.html) work too.
509     $(document).on("click", ".copy-btn", function () {
510         const $button = $(this);
511         const text = $($button.attr("data-copy-target")).text().trim();
512         if (!text) return;
513         copyText(text).then(() => {
514             showCopied($button);
515             showToast("Copied to clipboard!", "success");
516         }, () => showToast("Copy failed. Select the code and press Ctrl+C.", "danger"));
517     });
518
519     // The "copy" event also covers manual copying: select a code and press Ctrl+C (or use the menu).
520     $(document).on("copy", ".copy-snippet-code", () => showToast("Copied to clipboard!", "success"));
521 }
```

Figure 26. Actual source: js/script.js, copyText() and initCopyButtons().

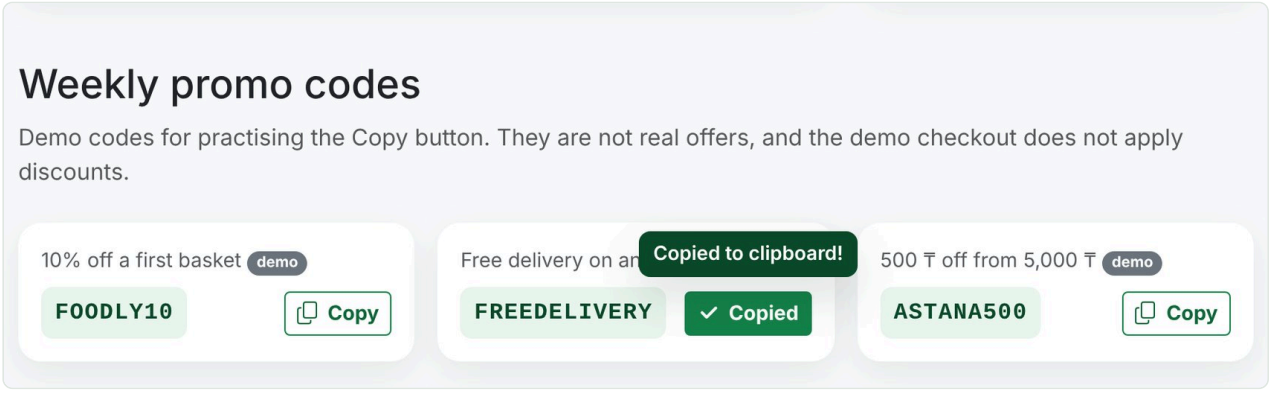


Figure 27. Home page: FREEDELIVERY copied, so the button shows a checkmark with “Copied” and the tooltip.

8. Copy to clipboard (continued)

js/script.js · lines 482-503

```
482 function showCopied($button) {
483     clearTimeout($button.data("resetTimer"));
484     $button.data("tooltip")?.remove();
485     $button.addClass("is-copied");
486     $button.find(".copy-icon").html(ICON_CHECK);
487     $button.find(".copy-label").text("Copied");
488
489     const $tooltip = $('<span class="copy-tooltip" aria-hidden="true">Copied to clipboard!</span>').appendTo("body");
490     const offset = $button.offset(); // position in page coordinates
491     const left = offset.left + ($button.outerWidth() - $tooltip.outerWidth()) / 2;
492     const maxLeft = $(window).width() - $tooltip.outerWidth() - 8;
493     $tooltip.css({ top: offset.top - $tooltip.outerHeight() - 10, left: Math.max(8, Math.min(left, maxLeft)) })
494         .hide().fadeIn(150);
495     $button.data("tooltip", $tooltip);
496
497     $button.data("resetTimer", setTimeout(() => {
498         $tooltip.fadeOut(200, () => $tooltip.remove());
499         $button.removeClass("is-copied");
500         $button.find(".copy-icon").html(ICON_COPY);
501         $button.find(".copy-label").text("Copy");
502     }, COPY_RESET_DELAY));
503 }
```

Figure 28. Actual source: js/script.js, showCopied(): checkmark, tooltip and reset.

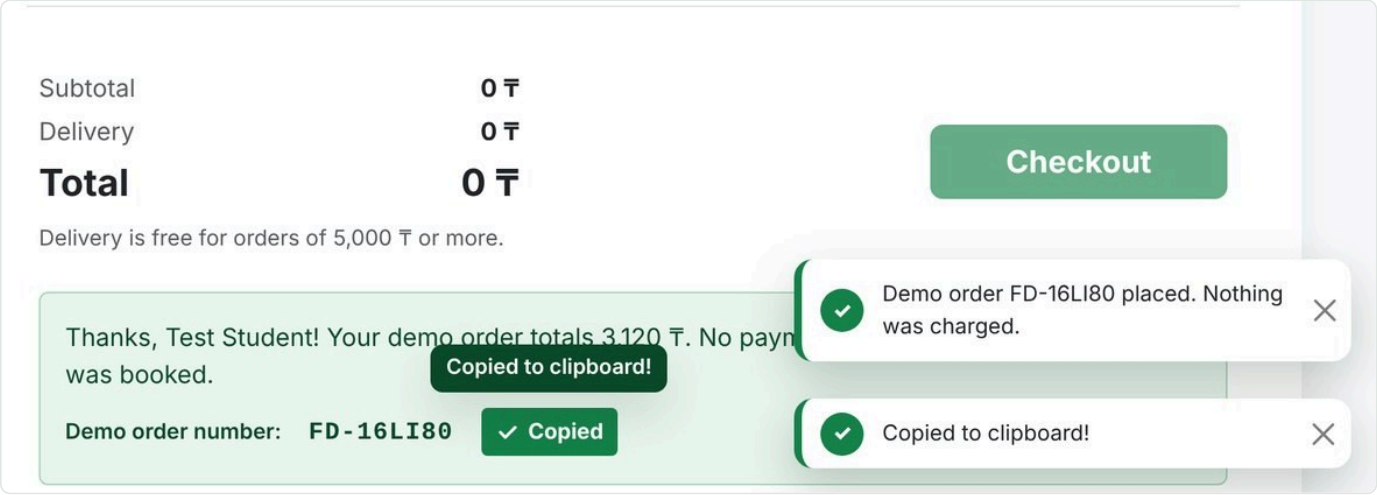


Figure 29. Cart page after a demo order: the order number FD-16LI80 copied, with the tooltip and two toasts.

The clipboard content was read back in the test (“FREEDELIVERY”, “FD-16LI80”). The promo codes are labelled demo and give no discount.

jQuery used: `.on('click')`, `.on('copy')`, `.attr()`, `.text()`, `.val()`, `.offset()`, `.outerWidth()`, `.fadeIn()/.fadeOut()`

9. Image lazy loading

1. Move each photo's URL from `src` to `data-src` and use a 1×1 transparent placeholder (13 images on Home, all product photos on Stores). Native `loading="lazy"` was removed.
2. On scroll/resize (throttled to 150 ms) compare `$img.offset().top` with `$(window).scrollTop()` + `height()`, with a 200 px margin.
3. Load visible images with `.attr('src', data-src)`, then fade them in; remove the handler when nothing is pending.
4. Hidden carousel slides load on Bootstrap's `slide.bs.carousel` event; `width/height` prevent layout shift; `<noscript>` is the fallback.

js/script.js · lines 530-567

```
530 function loadLazyImage($img) {
531     const source = $img.attr("data-src");
532     if (!source) return;
533     $img.removeAttr("data-src").addClass("is-loading")
534         // Bind before setting src, so a cached image cannot finish before we listen.
535         .one("load", () => {
536             $img.removeClass("is-loading").addClass("is-loaded").css("opacity", 0).animate({ opacity: 1 }, 400);
537         })
538         .one("error", () => $img.removeClass("is-loading"))
539         .attr("src", source);
540 }
541
542 function checkLazyImages() {
543     // Slides that are not active yet are display:none; the carousel event below loads them.
544     const $pending = $("img.lazy-img[data-src]").filter(function () {
545         return !$(this).closest(".carousel-item:not(.active)").length;
546     });
547     const $window = $(window);
548     if (!$pending.length) {
549         $window.off("scroll.lazy resize.lazy"); // everything loaded: stop listening
550         lazyHandlerBound = false;
551         return;
552     }
553     if (!lazyHandlerBound) {
554         $window.on("scroll.lazy resize.lazy", throttle(checkLazyImages, 150));
555         lazyHandlerBound = true;
556     }
557
558     const viewTop = $window.scrollTop();
559     const viewBottom = viewTop + $window.height();
560     $pending.each(function () {
561         const $img = $(this);
562         if (!$img.is(":visible")) return; // e.g. a filtered-out product card
563         const top = $img.offset().top;
564         const bottom = top + $img.outerHeight();
565         if (top < viewBottom + LAZY_MARGIN && bottom > viewTop - LAZY_MARGIN) loadLazyImage($img);
566     });
567 }
```

Figure 30. Actual source: js/script.js, loadLazyImage() and checkLazyImages().

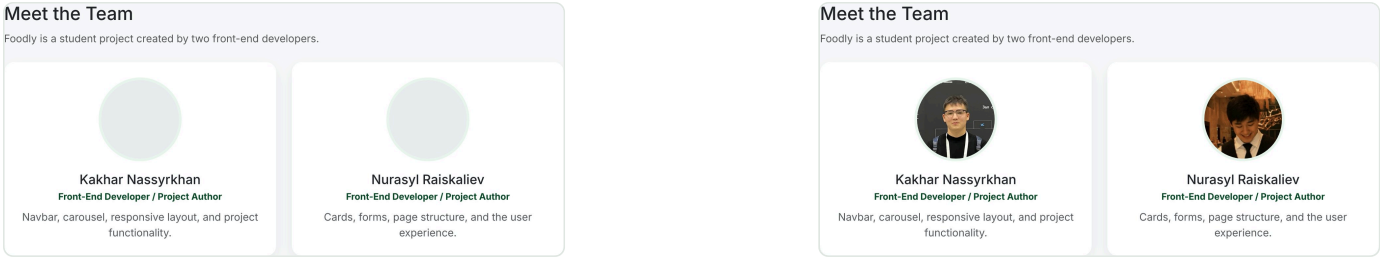


Figure 31. Team section on a deliberately slowed connection (4 s delay): placeholders while loading (left), photos faded in (right).

9. Image lazy loading (continued)

DOM of a team photo · before and after scrolling (captured)

```
BEFORE:
<img class="portrait-kakhar lazy-img" src="" data-src="images/nurasyl.jpg" alt="Kakhar Nassyrkhan wearing glasses at the hackalem.ai event" width="128" height="128">

AFTER:
<img class="portrait-kakhar lazy-img is-loaded" src="images/nurasyl.jpg" alt="Kakhar Nassyrkhan wearing glasses at the hackalem.ai event" width="128" height="128" style="opacity: 1;">
```

Figure 32. The same <img> before it entered the viewport (placeholder, data-src) and after loading.

Moment	Image files requested by the browser
Page load (top of Home)	fruits.jpg — only the first carousel slide
After scrolling to the bottom	fruits.jpg, magnum.jpg, small.jpg, galmart.jpg, nurasyl.jpg, kakhar.jpg
Carousel “Next”	next slides load on demand (8 → 6 waiting)
Everything loaded	scroll handler removed (`lazyHandlerBound === false`)



Figure 33. Carousel after “Next”: the incoming slide was loaded by the slide.bs.carousel handler.

jQuery used: `.offset()`, `.scrollTop()`, `.height()`, `.outerHeight()`, `.attr('src')`, `.removeAttr()`, `.one('load')`, `.animate()`, `.on()/.off('scroll.lazy')`

10. Verification and regression tests

Check	Recorded outcome
Independent re-check (this report)	62/62 checks passed on all six pages: Tasks 0-9, mobile 375 px
Console / runtime / HTTP errors	0 on every page and state checked
Project test suite (`tests/browser.cjs`)	175 passed, 0 failed (recorded in docs/TESTING.md, Chrome 156, Windows 11)
HTML / JavaScript	6 pages pass html-validate; 9 scripts pass `node --check` (docs/TESTING.md)
Accessibility	0 axe violations on 6 pages and 4 extra states (docs/TESTING.md)
Responsive widths	No horizontal overflow at 375 px (re-checked); 390/768/1440 px in the suite
Live website	jQuery 4.0.0 from the CDN, “jQuery is ready!”, no console errors (checked 4 Nov 2026)

What the re-check asserted (selection)

- T1 live filter: 8 cards -> 1 visible for "chick"
- T1 status text: "1 product shown"
- T1 clearing restores all 8 cards
- T2 list open with 6 suggestions (aria-expanded=true)
- T2 ArrowDown x2 -> catalog-suggestion-1; 6 <mark> highlights
- T2 Enter fills "Bread" and closes list
- T2 choice filters catalog
- T2 typed HTML never inserted
- T7 three stacked toasts (3)
- T7 toast text "Milk 1L added to your cart."
- T7 role=status
- T7 max 4 active toasts
- T7 toasts fade out automatically
- T1 compare: dairy 3 rows -> 1 for "butter" ("1 product shown")
- T9 13 managed images, 13 waiting in data-src, native lazy=0, noscript=13
- T9 team photos not requested at load (loaded: fruits.jpg)
- T4 bar empty at top (0.0px)
- T5 final counters 3 / 24 / 60 min / 5,000 ₸
- T5 counters start at 0 below the fold (0 / 0 / 0 min / 0 ₸)
- T5 mid-animation 1 / 8 / 20 min / 1,682 ₸
- T5 counter never restarts
- T4 bar ≈ 50.0% half-way
- T4 bar full at bottom (100.0%)
- T9 scrolling loaded team photos (5 new image requests)
- T9 carousel Next loads slides (8 -> 6 waiting)
- T3 "Found 5 matches", 2 answers auto-opened
- T3 link hrefs untouched
- T3 special characters escaped (no error)

The re-check ran in Chromium (Playwright) on a local copy of the submitted source. jQuery was served from the identical 4.0.0 file (same SRI hash) because the test machine had no CDN access; the live site was checked separately with the real CDN.

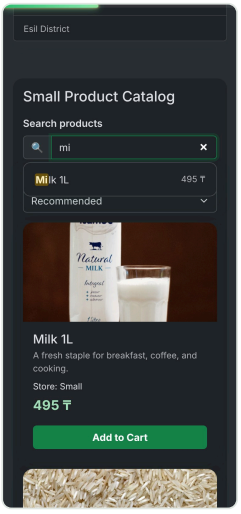


Figure 34. Mobile 375 px in night theme: autocomplete on the Small catalog.

11. Commit, push and deployment

1. Run the browser suite, HTML validation and JavaScript syntax checks.
2. Commit the implementation to the shared repository and push `main`; GitHub Pages builds from the repository root.
3. Wait for the Pages workflow to succeed, then open the live site and verify the new jQuery features.

Implementation commit: 2b84633
Pages result: built successfully; live URL checked on 4 November 2026.
Live check: jQuery 4.0.0 loaded from code.jquery.com, console shows “jQuery is ready!” with no errors, counters finish at 3 / 24 / 60 min / 5,000 ₸, scroll bar and 13 lazy images present.

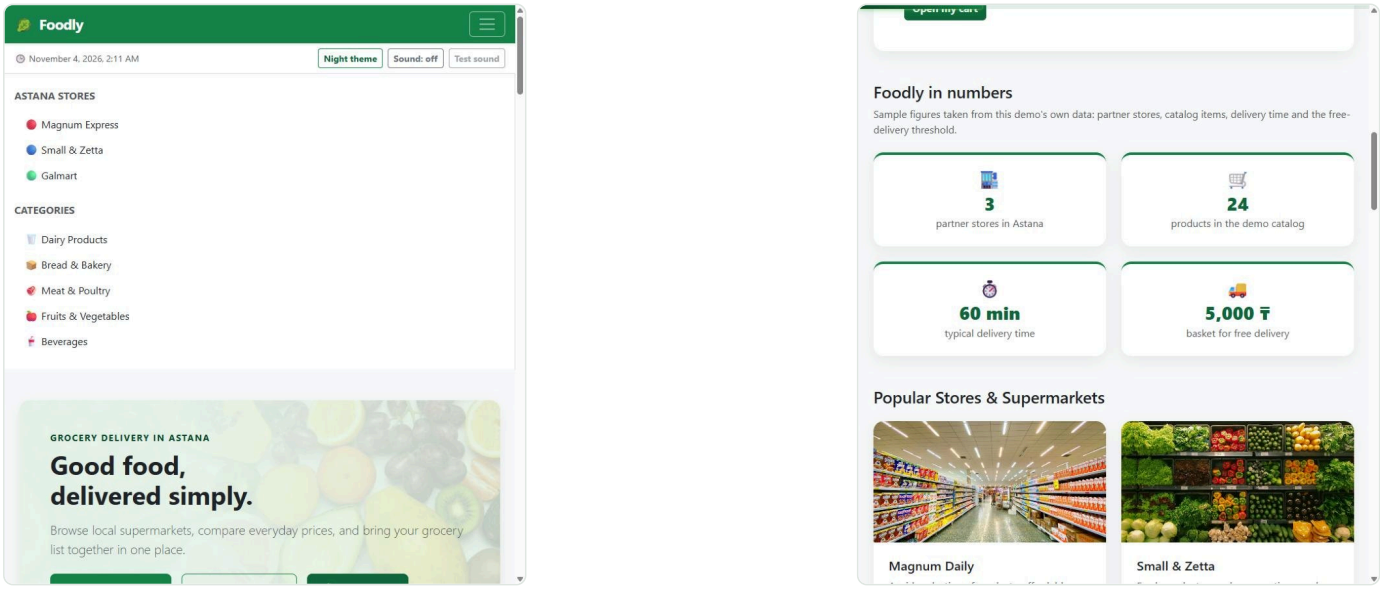


Figure 35. The public GitHub Pages site opened in a browser on 4 November 2026: homepage (left), live counters and progress bar (right).

LIVE WEBSITE
https://vuus1d.github.io/Food-Delivery-Platform-KN/
GITHUB REPOSITORY
https://github.com/vuus1d/Food-Delivery-Platform-KN

Deployment workflow: View successful GitHub Pages run

Reflection and defense preparation

Challenges and resolutions

Two handlers on one search field. app.js already had a vanilla filter. It was removed; one jQuery `filterCatalog()` is called by app.js after rendering a store.

Highlighting could break links. Replacing inside `.html()` would also change `href="compare.html"`. Only text nodes are wrapped, and the original HTML is restored from `.data()`.

fadeIn and the hidden attribute. jQuery cannot show an element with `[hidden]`. The code calls `.hide()`, removes `hidden`, then `.fadeIn()`.

Hidden carousel slides. `display:none` slides report offset 0, so they load on Bootstrap's `slide.bs.carousel` event instead of on scroll.

Spinner and existing validation. The spinner starts in the success callback, so invalid forms never show it; checkout blocks closing the dialog during the wait.

Toast limit with fast clicks. Toasts that were fading out were still counted. They are marked `.is-closing` and excluded.

Bootstrap blocked on Windows. Git converted vendor files to CRLF, so the SRI hash failed. A `.gitattributes` file keeps them byte-exact.

What to explain at the defense

Why `$(document).ready()` is needed for a script in <head>; `keyup` vs `input`; how `.filter()` works; why the regex is escaped and why `.text()` is safer than `.html()`; how the scroll percentage is calculated; how `.animate()` tweens a plain object; why the submit button is disabled; how `.offset()` decides when an image loads; the Clipboard API and its fallback. Use docs/DEFENSE-RU.md and docs/ASSIGNMENT-6-NOTES.md for the demo order and answers.

Submission checklist

Submit the PDF report and project ZIP separately in LMS. Every team member must submit. Check the deadline in lms.astanait.edu.kz and attend the practice-lesson defense. The LMS deadline and submission status were not available and are not claimed as completed.

Source of requirements

Assignment 6 jQuery.pdf: Task 0 (Setup), Part 1 (jQuery Search), Part 2 (UX Engagement Elements), Part 3 (Web App Functionality), Part 4 (Deploy), submission rules and evaluation criteria. Project code and screenshots are included in the accompanying ZIP.